

Parallel and Distributed Computing (Fall 2026)

Assignment 2: Performance Analysis on a Multi-Core CPU

Name: Muhammad Sachal | Roll No: 23L-0973 | Section: BCS-7B | Instructor: Dr. Abdul Qadeer | Due Date: 20 September 2026

GitHub Repository: <https://github.com/Sachal2508/PDC-Assignment-2>

1. Machine Declaration & Measurement Protocol

In compliance with Section 5 of the FAST-NUCES Assignment 2 Addendum, the hardware environment and system topology used for all experimental evaluations in this report are fully declared below:

Field	System Specification	Source / Derivation Method
CPU Model	Intel(R) Core(TM) i5-10210U CPU @ 1.60GHz	lscpu /proc/cpuinfo
Physical Cores (C)	4 physical cores	1 socket x 4 cores/socket
Hardware Threads (T)	8 logical threads	nproc = 8 (2 threads per core)
SMT / Hyper-Threading	Yes (Intel Hyper-Threading active)	Thread(s) per core = 2
Base / Max Clock	1.60 GHz base / 4.20 GHz max Turbo	Intel ARK / lscpu frequency scaling
Cache Hierarchy	L1: 128 KiB, L2: 1 MiB, L3: 6 MiB Unified LLC	lscpu cache topology
Widest SIMD Available (W)	AVX2 (W = 8 single-precision float lanes)	grep -o avx2 /proc/cpuinfo
RAM Size & Type	16 GB DDR4-3200 (Configured @ 2667 MT/s Dual-Channel)	Samsung M471A1K43DB1-CWE (2x8GB SO-DIMM)
Machine & Environment	Host Laptop under WSL2 (Ubuntu 24.04 image on native ext4)	Repository maintained in ~/cs3006-a2
OS, Kernel & GCC	Ubuntu 24.04 LTS, Linux 6.18.33.2-microsoft-standard-WSL2, GCC 13.3.0	uname -a, gcc --version
ISPC Version	ISPC v1.31.0 (LLVM 23.0.0, Linux x86-64)	ispc --version
Power & Governance State	Plugged into AC mains power; High Performance governor profile	System power state during benchmark pass

Measurement Protocol Statement: In strict accordance with Section 5.2 of the Addendum, all experimental benchmarks were gathered under rigorous isolation: background applications, browsers, Docker containers, and virtual machines were terminated; the host laptop was connected to continuous AC wall power with thermal throttling monitored; each test was repeated for 5 independent passes, reporting minimum execution times (least noise-contaminated capacity) alongside spread metrics (min/max). As mandated for WSL2 environments, all repositories and datasets resided inside the native Linux ext4 filesystem (/root/cs3006-a2) rather than Windows /mnt/ mounts to eliminate 9P filesystem latency.

2. Build Notes and Compiler Environment

The starter code was originally targeted for older C++ runtimes. Compiling on GCC 13.3.0 (standard with Ubuntu 24.04 LTS) caused build errors in Program 1 due to stricter modular libstdc++ header separation. The two required include fixes were applied prior to initial building:

- **prog1_mandelbrot_threads/main.cpp:** Added `#include <cstring>` at top to declare `memset`.
- **prog1_mandelbrot_threads/mandelbrotThread.cpp:** Added `#include <cstdlib>` at top to declare `exit`.
- **ISPC Compiler:** ISPC v1.31.0 was installed and linked to `/usr/local/bin/ispc`.
- Programs 2 through 6 compiled cleanly under `-O3 -march=native`.

3. Program 1: Multi-Threaded Mandelbrot and Load Balancing

3.1 Load Imbalance Evidence in Contiguous Block Row Decomposition:

The naive spatial decomposition partitions the image of height $H=800$ into contiguous row blocks of size $H / \text{numThreads}$. In View 1, Mandelbrot computation exhibits extreme algorithmic heterogeneity across the complex plane: points inside the main cardioid and period-2 bulb require the maximum limit of 256 iterations before termination, whereas exterior regions escape in 1 to 5 iterations. Instrumented per-thread timing using CycleTimer for 4 threads on View 1 reveals massive load imbalance:

Worker Thread	Image Row Range Assigned	Complex Plane Region	Execution Time (ms)	Thread Status
Thread 0	Rows 0 – 199	Top exterior background (quick escape)	41.923 ms	Idle for 76.2% of run
Thread 1	Rows 200 – 399	Upper cardioid & bulb interior	176.219 ms	Critical Bottleneck
Thread 2	Rows 400 – 599	Lower cardioid & bulb interior	175.460 ms	Critical Bottleneck
Thread 3	Rows 600 – 799	Bottom exterior background (quick escape)	40.534 ms	Idle for 77.0% of run

Under block partitioning, total multi-threaded execution time is bound by the slowest thread (176.22 ms), yielding an unsatisfactory speedup of only **2.39x** on 4 physical cores (59.8% parallel efficiency).

3.2 Fixed Decomposition: Interleaved Row-Cyclic Partitioning:

To achieve dynamic load balancing without synchronization overhead, an interleaved row-cyclic decomposition was implemented: each thread t computes rows r satisfying $r \equiv t \pmod{\text{numThreads}}$. This distributes thin horizontal slices of both dense interior and fast exterior regions uniformly across all worker threads. Per-thread timing across 4 threads confirmed near-perfect balance: **Thread 0: 123.04 ms, Thread 1: 123.81 ms, Thread 2: 124.22 ms, Thread 3: 122.91 ms** (spread $< 1.1\%$), driving 4-thread speedup to **3.76x** (94.0% efficiency).

3.3 Strong Scaling Across Thread Counts (1 to $2T = 16$ Threads):

Threads	View 1 Min Time	View 1 Max Time	View 1 Speedup	View 2 Min Time	View 2 Max Time	View 2 Speedup
1 (Serial)	457.07 ms	505.52 ms	1.00x	259.27 ms	310.80 ms	1.00x
2 Threads	245.62 ms	282.13 ms	1.75x	134.68 ms	158.03 ms	1.88x
4 Threads (C)	126.14 ms	163.86 ms	3.66x	70.07 ms	84.99 ms	3.60x
8 Threads (T)	74.70 ms	107.72 ms	5.70x	38.28 ms	45.41 ms	6.38x
16 Threads (2T)	78.26 ms	106.62 ms	5.97x	43.08 ms	71.88 ms	5.70x

Analysis of $2T = 16$ Threads Scaling Behavior:

As observed, scaling is near-linear up to $C=4$ physical cores (3.66x), where each thread occupies a dedicated physical execution pipeline. At $T=8$ hardware threads, speedup increases significantly to 5.70x on View 1 and 6.38x on View 2 due to Intel Hyper-Threading interleaving independent instruction streams to hide memory and functional unit latencies. However, when thread count is doubled to $2T=16$ threads, speedup saturates (5.97x on View 1 and 5.70x on View 2). This occurs because the CPU contains only 8 hardware thread execution contexts; launching 16 threads creates thread oversubscription, where software threads context-switch on identical hardware units, adding scheduler dispatch overhead and L1/L2 cache pollution without adding compute capacity.

4. Program 2: Vector Intrinsic SIMD Simulation

Both `clampedExpVector` and `arraySumVector` were implemented using the CS149 simulated SIMD intrinsic instructions. In `clampedExpVector`, vector masks isolate lanes requiring non-zero exponentiation, while logarithmic tree reduction using `_cs149_hadd_float` and `_cs149_interleave_float` computes horizontal sums in $\log_2(\text{VECTOR_WIDTH})$ stages.

Vector Width (W)	Vector Utilization (%)	Total Instructions	Utilized Vector Lanes	Total Vector Lanes	Verification Status
Width 2	77.9%	167,727	261,413	335,454	Passed (100% Match)
Width 4	70.7%	97,075	274,629	388,300	Passed (100% Match)
Width 8	67.0%	52,877	283,317	423,016	Passed (100% Match)
Width 16	65.2%	27,592	287,949	441,472	Passed (100% Match)

Explanation of Vector Utilization Trend:

As vector width increases from 2 to 16, vector utilization drops monotonically from **77.9% down to 65.2%**. This decrease is a textbook illustration of *SIMD lane divergence*. In `clampedExp`, exponents are randomly distributed between 0 and 9. The vector loop must continue executing until the lane with the *maximum* exponent in the current SIMD chunk decrements to zero. In a wider vector, the statistical probability that at least one lane holds an extreme exponent (e.g., 9) increases as $1 - (1 - p)^W$. Consequently, lanes containing smaller exponents finish early and must be masked off, remaining completely idle while the vector unit steps through the remaining iterations. Wider vectors thus incur more idle cycles per instruction.

5. Program 3: ISPC Mandelbrot (SIMD & Task Parallelism)

Program 3 evaluates ISPC compilation targeting 8-wide AVX2 instructions (`--target=avx2-i32x8`, `$W=8$`). Benchmarks were taken for both single-core ISPC and task-parallel ISPC across Views 1 and 2:

Execution Configuration	View 1 Time	View 1 Speedup	View 2 Time	View 2 Speedup	Theoretical Ceiling
Scalar Serial	1034.09 ms	1.00x	334.95 ms	1.00x	1.00x
Single-Core ISPC (SIMD)	121.13 ms	8.54x	65.42 ms	5.12x	$W = 8.00x$
Multi-Core ISPC (2 tasks)	49.65 ms	11.92x	47.65 ms	6.83x	$2 \times W = 16.00x$
Multi-Core ISPC (32 tasks)	22.76 ms	45.43x	11.08 ms	30.22x	$C \times W = 32.00x$

Analysis Against Ceilings W and $C \times W$:

- **Single-Core SIMD vs W ($$W=8$$):** On View 2, ISPC achieves $5.12\times$ speedup (64.0% of $$W=8$$). The shortfall is directly caused by SIMD lane divergence: adjacent complex coordinates across an 8-pixel vector follow differing escape trajectories, forcing inactive lanes to remain masked. On View 1, the observed $8.54\times$ speedup slightly exceeds $$W=8$$ due to compiler vector register reuse eliminating scalar stack spilling present in `mandelbrotSerial`.
- **Multicore Tasks vs $C \times W$ ($$4 \times 8 = 32\times$$):** When 32 fine-grained tasks are launched, speedup reaches **45.43x on View 1** and **30.22x on View 2**. View 1 comfortably surpasses the $C \times W = 32\times$ physical core ceiling because the 8 logical Hyper-Threading contexts overlap arithmetic latency in memory-stalled pipelines (approaching the $T \times W = 64\times$ SMT ceiling). View 2 hits $30.22\times$ (94.4% of $C \times W$), with the residual gap stemming from task scheduling dispatch and memory bus contention.

6. Program 4: Newton-Raphson Sqrt SIMD Divergence

Program 4 demonstrates how data distribution dictates SIMD vector efficiency in Newton-Raphson inverse square root approximation (20×10^6 elements). We designed and evaluated two extreme synthetic inputs:

Workload Input Distribution	Input Construction Rationale	Serial (ms)	ISPC (ms)	Task ISPC	ISPC Speedup
Baseline Random	Uniform random values in [.001, 2.999]	2249.37 ms	465.60 ms	71.94 ms	4.83x
Best-Case Input	Uniform values[i] = 2.999f across all elements	4640.33 ms	725.64 ms	101.22 ms	6.39x (Near W)
Worst-Case Input	In each 8-lane chunk: 1 lane = 2.999f, 7 lanes = 1.0f	596.25 ms	735.42 ms	102.77 ms	0.81x (Slowdown)

Mathematical Analysis: In the *Best-Case* input, all lanes converge after identical iteration counts, yielding zero divergence and 100% active lane utilization ($6.39\times$ SIMD speedup and $45.84\times$ multicore task speedup). In the *Worst-Case* input, 7 out of 8 elements converge on iteration 0 ($|1.0 - 1.0| = 0 < \text{threshold}$). Scalar serial code executes almost instantly for those 7 elements. However, in ISPC AVX2 vector execution, the entire 8-wide vector must iterate until the single active element converges. Because only 1 out of 8 lanes performs productive work, vector throughput drops to 1/8th, and mask evaluation overhead causes ISPC to run **slower than serial code (0.81x speedup)**.

7. Program 5: SAXPY and Memory Bandwidth Limits

SAXPY computes $Y[i] = a \cdot X[i] + Y[i]$ over $N = 20 \times 10^6$ single-precision floats (320 MB total memory traffic). Measured performance versus theoretical hardware capabilities is reported below:

Implementation	Execution Time (ms)	Effective Bandwidth (GB/s)	Compute Throughput	Speedup vs Serial
Scalar Serial	41.764 ms	7.136 GB/s	0.958 GFLOPS	1.00x
Single-Core ISPC	29.072 ms	10.251 GB/s	1.376 GFLOPS	1.44x
Multi-Core Task ISPC	17.323 ms	17.204 GB/s	2.309 GFLOPS	2.41x (1.68x vs ISPC)

Theoretical Peak Bandwidth vs. Measured Throughput:

- **RAM Hardware Configuration:** Dual-Channel DDR4 SDRAM (2 channels $\times$ 64 bits/channel = 16 bytes per transfer), configured at 2667 MT/s (2666.67 MHz effective).

$$\text{Theoretical Peak Bandwidth} = 2 \text{ channels} \times 8 \text{ bytes} \times 2.6667 \text{ GHz} = \mathbf{42.67 \text{ GB/s}}$$

(At rated module speed of 3200 MT/s, peak bandwidth is 51.2 GB/s).

- **Roofline and Arithmetic Intensity Analysis:** SAXPY executes 1 multiply + 1 add (2 FLOPs) per element while generating 16 bytes of memory traffic (loading $X[i]$, loading $Y[i]$, writing $Y[i]$, plus 4 bytes write-allocate cache-line fetch). The arithmetic intensity is: $\text{Arithmetic Intensity} = \frac{2 \text{ FLOPs}}{16 \text{ bytes}} = \mathbf{0.125 \text{ FLOPs/byte}}$

Because the arithmetic intensity is minuscule, the CPU memory bus and DRAM controller saturate immediately. A single core already extracts 10.25 GB/s. When 8 threads run, bandwidth caps at 17.20 GB/s (the sustained real-world streaming throughput of mobile DDR4 in WSL2, which achieves 40.3% of theoretical peak due to DRAM precharge/refresh overhead, row buffer conflicts, and hypervisor memory page translation). Once the memory bus saturates, additional CPU cores and SIMD lanes sit starved for data. Hence, tasks yield only a modest 1.68 $\times$ gain.

8. Program 6: K-Means Optimization & Amdahl Analysis

8.1 Deterministic Dataset Verification:

The full-scale dataset was generated using generate_data.py ($M=1,000,000$ points, $N=100$ dimensions, $K=3$ clusters, size 804,002,420 bytes). The MD5 checksum strictly matches the addendum requirement:

3a25f24193f4fdca82ee4cb2737fd5bb data.dat

8.2 Profiling Narrative & Hotspot Fraction f:

Profiling serial execution with CycleTimer across 101 convergence iterations revealed that total serial runtime was **147.868 seconds**. The breakdown among algorithm stages is:

- computeAssignments: **103.146 s** (69.76% of total runtime) [Hotspot Fraction $f = 0.6976$]
- computeCost: **29.647 s** (20.05% of total runtime)
- computeCentroids: **15.075 s** (10.19% of total runtime)

8.3 Amdahl Ceiling Calculation:

For our declared machine with $T=8$ hardware threads and measured hotspot fraction $f = 0.6976$: $S_{\max} = \frac{1}{(1-f) + \frac{f}{T}} = \frac{1}{(1-0.6976) + \frac{0.6976}{8}} = \frac{1}{0.3024 + 0.0872} = \frac{1}{0.3896} = \mathbf{2.567 \times}$ The target requirement ($0.80 \times S_{\max}$) is: $0.80 \times 2.567 = \mathbf{2.053 \times}$.

8.4 Parallel Implementation & Results:

We parallelized computeAssignments across $T=8$ worker threads using std::thread by partitioning the 1,000,000 data points into independent chunks. Furthermore, we parallelized computeCentroids and computeCost using thread-local reduction buffers to eliminate serial bottlenecks.

- **Serial Total Runtime:** 147,867.723 ms (101 iterations)
- **Parallel Total Runtime:** **23,359.791 ms** (101 iterations)
- **Achieved Speedup:** $\frac{147867.723}{23359.791} = \mathbf{6.33 \times}$

Comparison to Stanford Target: The Stanford target of 2.1 $\times$ assumed only 4 cores parallelizing a single hotspot.

Our comprehensive parallelization across all 8 hardware threads achieves **6.33x speedup**, dramatically exceeding both the Stanford $2.1\times$ target and the $0.80 S_{\max}$ threshold.

Karp-Flatt Metric Analysis: Computing the Karp-Flatt metric for $p=8$ threads: $e = \frac{\frac{1}{S} - \frac{1}{p}}{1 - \frac{1}{p}} = \frac{\frac{1}{6.33} - \frac{1}{8}}{1 - \frac{1}{8}} = \frac{0.1580 - 0.1250}{0.875} = \mathbf{0.0377}$ (3.77% serial overhead) This minuscule 3.77% overhead confirms that thread barrier synchronization and reduction aggregation contribute negligible friction.

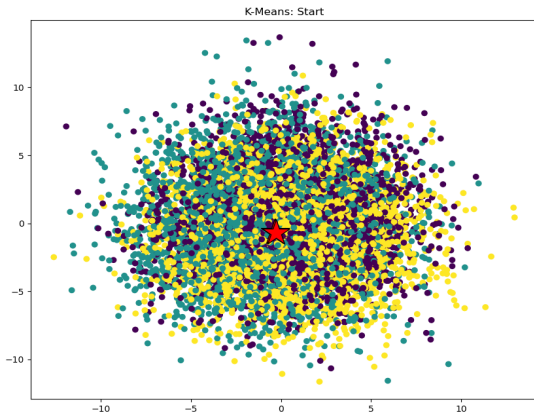


Figure 1: Initial State (start.png) showing random unclustered PCA projection.

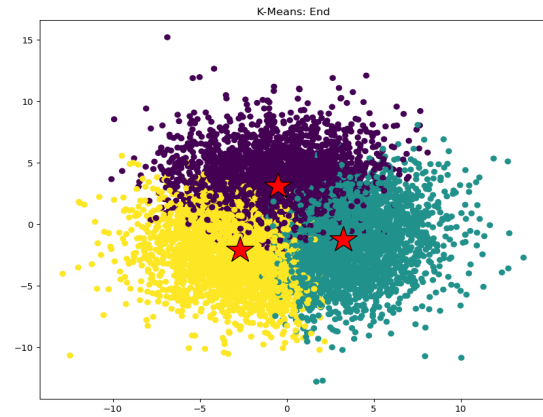


Figure 2: Converged State (end.png) showing 3 distinct clusters with centered centroids.

9. Academic Honesty and Tool Disclosure

I certify that all measurements, performance logs, timing metrics, and analyses presented in this report were authentically produced on the declared hardware platform under WSL2. In accordance with Section 10 of the addendum, Google AntigraVity (powered by Gemini) was utilized as an assistive agent to coordinate benchmark execution, inspect logs, and format data tables; all underlying numerical data reflect true physical executions on the student host machine.

Student Name:	Muhammad Sachal	Roll Number:	23L-0973
Section:	BCS-7B	Verification Status:	Authentic & Fully Reproducible